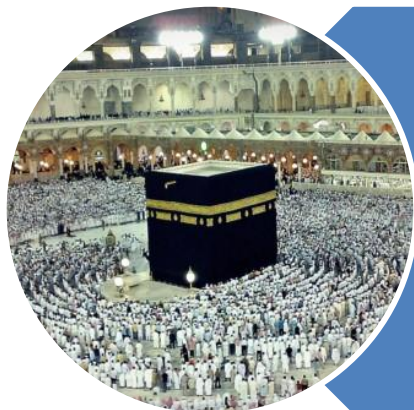


رَبِّ اشْرَحْ لِي صَدْرِي وَيَسِّرْ لِي أَمْرِي وَاحْلُلْ عُقْدَةً مِنْ لِسَانِي يَفْقَهُوا قَوْلِي



O my Lord! Expand for me my chest [with assurance] and ease for me my task and untie the knot from my tongue that they may understand my speech. **(Quran 20 : 25-28)**

Logarithmic Sorting

Quick Sort

Sorting: Quick Sort

- Quick Sort
 - Most common sort used in practice
 - Why?
 - cuz it is usually the quickest in practice!
 - Quick Sort uses two main ideas to achieve this efficiency:
 - 1) The idea of making partitions
 - 2) Recursion
 - Let's look at the partition concept...

Sorting: Quick Sort

- Quick Sort – Partition
 - A partition works as follows:
 - Given an array of n elements
 - You must manually select an element in the array to partition by
 - You must then compare ALL the remaining elements against this element
 - If they are greater,
 - Put them to the “right” of the partition element
 - If they are less,
 - Put them to the “left” of the partition element

Sorting: Quick Sort

- Quick Sort – Partition
 - A partition works as follows:
 - Once the partition is complete, what can we say about the position of the partition element?
 - We can say (we KNOW) that the partition element is in its CORRECTLY sorted location
 - In fact, after you partition the array, you are left with:
 - all the elements to the left of the partition element, in the array, that still need to be sorted
 - all the elements to the right of the partition element, in the array, that still need to be sorted
 - And if you sort those two sides, the entire array will be sorted!

“No Bonus” marks for this course anymore

Sorting: Quick Sort

- Quick Sort
 - Partition:
 - Essentially breaks down the sorting problem into two smaller sorting problems
 - ...what does that sound like?
 - Code for Quick Sort (at a real general level):
 - 1) Partition the array with respect to a random element
 - 2) Sort the left part of the array using Quick Sort
 - 3) Sort the right part of the array using Quick Sort
 - Notice there is no “merge” step like in Merge Sort
 - at the end, all elements are already in their proper order

Sorting: Quick Sort

- Quick Sort
 - Code for Quick Sort (at a real general level):
 - 1) Partition the array with respect to a random element
 - 2) Sort the left part of the array using Quick Sort
 - 3) Sort the right part of the array using Quick Sort
 - Quick Sort is a recursive algorithm:
 - We need a base case
 - A case that does NOT make recursive calls
 - Our base case, or terminating condition, will be when we sort an array with only one element
 - We know the array is already sorted!

C

P

C

S

2

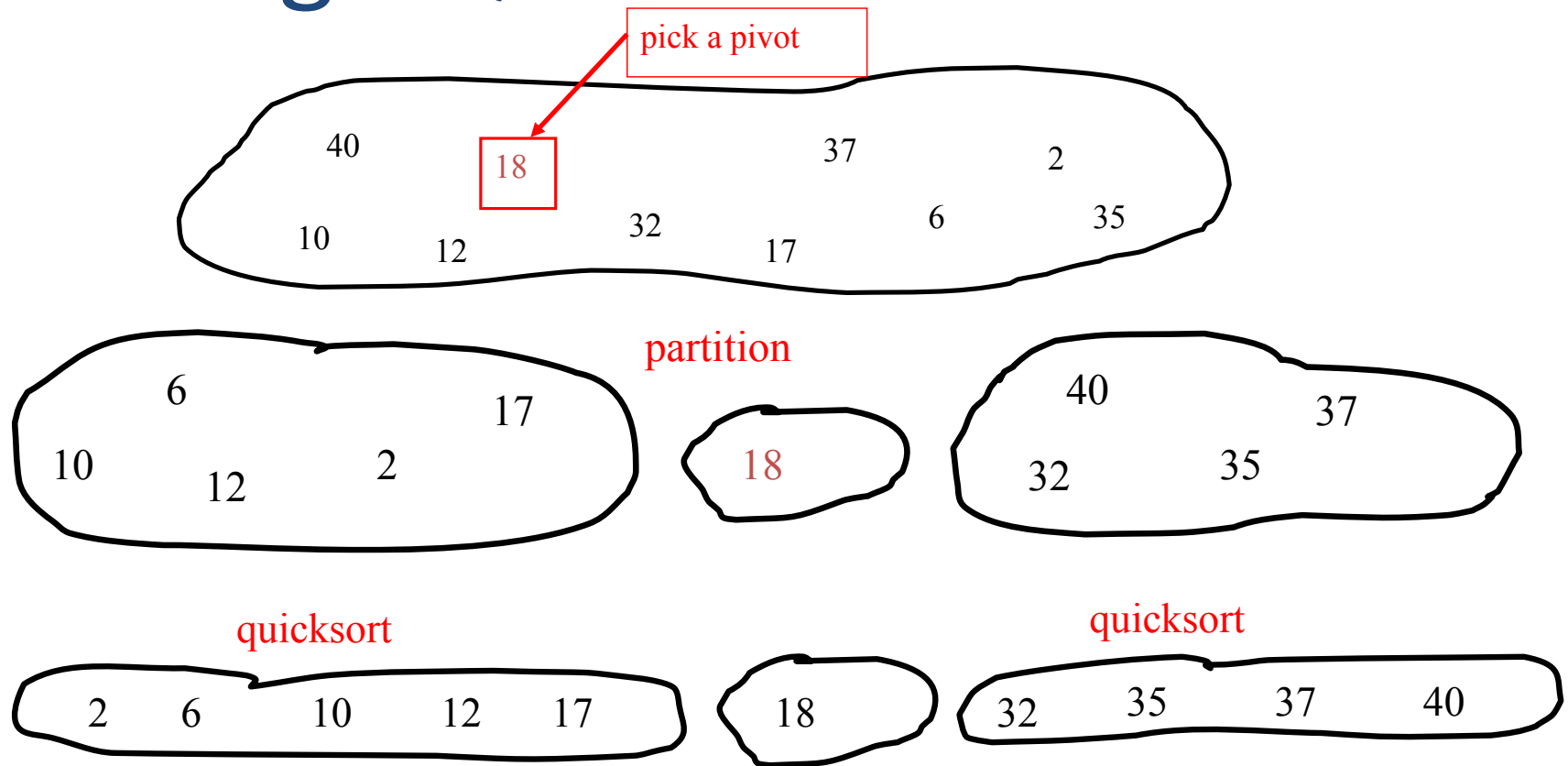
0

4

Sorting: Quick Sort

- Let S be the input set.
- 1. If $|S| = 0$ or $|S| = 1$, then return.
- 2. Pick an element v in S . Call v the partition element.
- 3. Partition $S - \{v\}$ into two disjoint groups:
 - $S_1 = \{x \in S - \{v\} \mid x \leq v\}$
 - $S_2 = \{x \in S - \{v\} \mid x \geq v\}$
- 4. Return { quicksort(S_1), v , quicksort(S_2) }

Sorting: Quick Sort



This is the final result. There is no need for a Merge/Combine step.
Each value is already in its correctly sorted position!

Sorting: Quick Sort

- Consider the following list of values that we want to partition

5	3	6	9	2	4	7	8
---	---	---	---	---	---	---	---

- Let us assume for the time being that we will partition based on the first element in the array
- The algorithm will partition these elements “in-place”

0	1	2	3	4	5	6	7
2	3	4	5	9	6	7	8

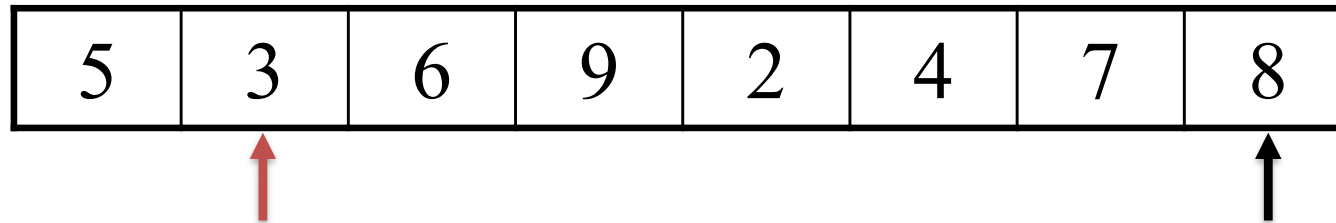
↑ ↑
4 x

loop ($x < 4$)

1. حرك x الى ان تكون قيمتها اكبر من البارتين
2. حرك y الى ان تكون قيمتها اصغر من اوتساوي البارتين
3. طالما $y < x$ سوي swap بينهما

سوي swap بين y والبارتين

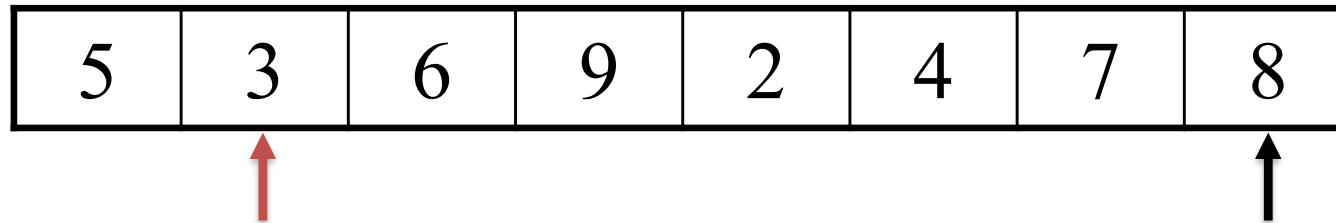
Sorting: Quick Sort



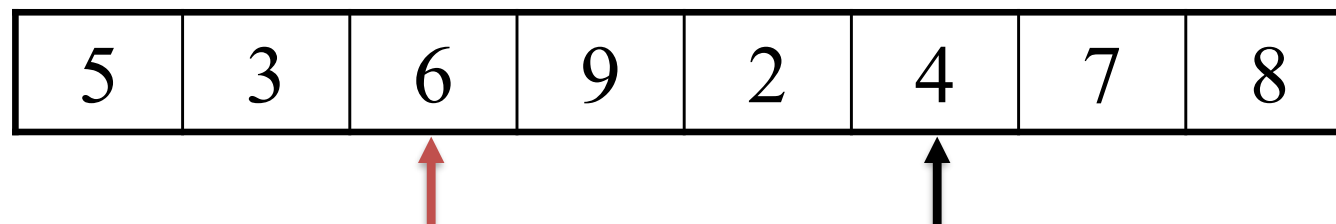
○ Here's how the partition will work:

- Start two counters, one at index one and one at index 7
 - The last element in the array
- Advance the left counter forward until an element greater than the partition element is encountered
- Advance the right counter backwards until a value less than the pivot is encountered

Sorting: Quick Sort



○ After these two steps are performed, we have:



Sorting: Quick Sort

5	3	6	9	2	4	7	8
---	---	---	---	---	---	---	---

↑
↑

- We know that these two elements are on the “wrong” side of the array ...so SWAP them!

5	3	4	9	2	6	7	8
---	---	---	---	---	---	---	---

↑
↑

Sorting: Quick Sort

5	3	4	9	2	6	7	8
---	---	---	---	---	---	---	---

↑
↑

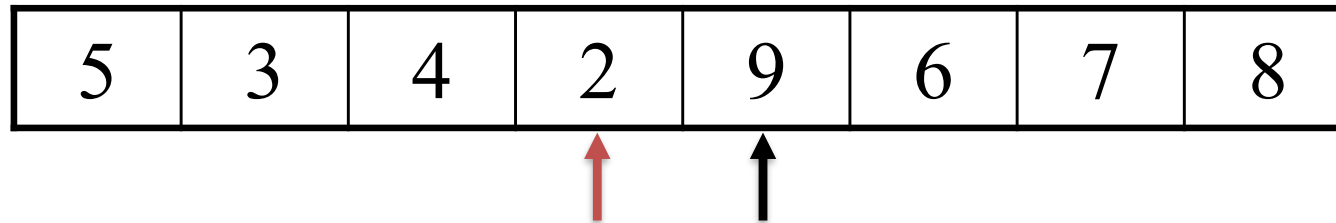
○ Then SWAP as before:

5	3	4	2	9	6	7	8
---	---	---	---	---	---	---	---

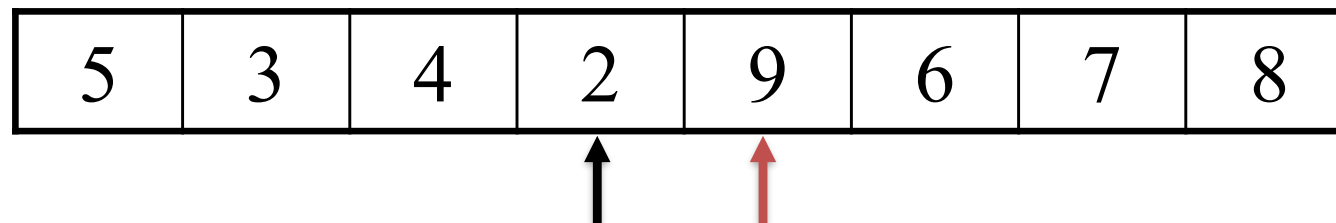
↑
↑

- At some point, the counters will cross over each other

Sorting: Quick Sort

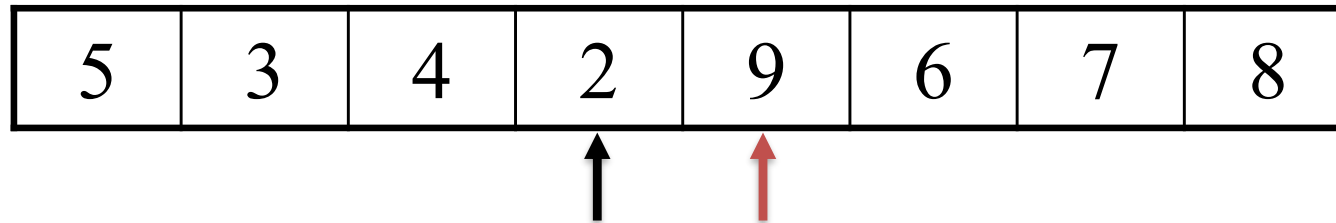


- Again, advance the pointers as before

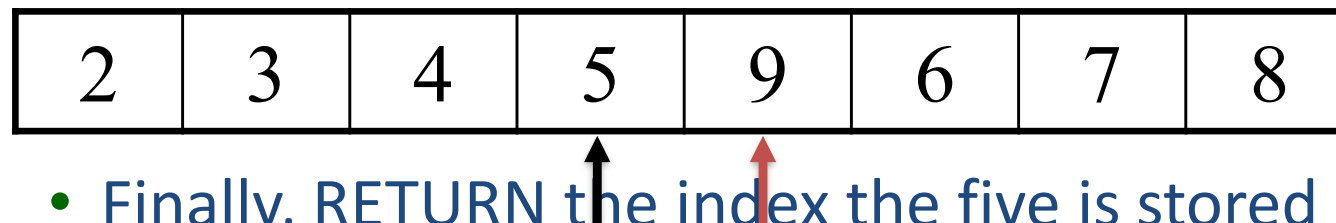


- So we see that the counters crossed over each other

Sorting: Quick Sort



- Now, SWAP the value stored in the original right counter (black arrow) with the partition element



- Finally, RETURN the index the five is stored in (the right pointer) to indicate where the partition element ended up

C

P

C

S

2

0

4

Sorting: Quick Sort

- Choosing a Partition Element
 - first element
 - bad if input is sorted or in reverse sorted order
 - bad if input is nearly sorted
 - variation: particular element (e.g. middle element)
 - random element
 - You could get lucky and choose the middle element
 - You could be unlucky and choose the smallest or greatest element
 - Resulting in a partition with ZERO elements on one side
 - median of three elements
 - choose the median of the left, right, and center elements

C

P

C

S

2

0

4

Sorting: Quick Sort

- median of three elements
 - choose the median of the left, right, and center elements
 - There is extra expense with this method
 - Picking three values
 - Doing three comparisons
 - But if the array is large, doing this little extra work will be small compared to the gains of a better partition
- You could also pick the median of 5 or 7 elements
 - The more you pick the better partition you get

Quick Sort - **Tracing**

Index	0	1	2	3	4	5	6	7
Initial	23	20	49	26	17	22	25	29
Loop 1								
Loop 2								
Loop 3								

Quick Sort - **Tracing**

Index	0	1	2	3	4	5	6	7
Initial	23	20	<u>49</u>	26	17	<u>22</u>	25	29
Loop 1								
Loop 2								
Loop 3								

Quick Sort - **Tracing**

Index	0	1	2	3	4	5	6	7
Initial	23	20	49	26	17	22	25	29
Loop 1	23	20	<u>22</u>	26	17	<u>49</u>	25	29
Loop 2								
Loop 3								

Quick Sort - **Tracing**

Index	0	1	2	3	4	5	6	7
Initial	23	20	49	26	17	22	25	29
Loop 1	23	20	<u>22</u>	26	17	<u>49</u>	25	29
Loop 2	23	20	22	<u>17</u>	<u>26</u>	49	25	29
Loop 3								

Quick Sort - **Tracing**

Index	0	1	2	3	4	5	6	7
Initial	23	20	49	26	17	22	25	29
Loop 1	23	20	<u>22</u>	26	17	<u>49</u>	25	29
Loop 2	23	20	22	<u>17</u>	<u>26</u>	49	25	29
Loop 3	17	20	22	23	26	49	25	29

Quick Sort

Analysis

C

P

C

S

2

0

4

Quick Sort - **Analysis**

- This is more difficult to do than Merge Sort
 - Why?
 - With Merge Sort, we knew that our recursive calls always had equal sized inputs
 - Remember: we would split the array of size n into two arrays of size $n/2$ (so the smaller arrays were always the same size)
- How is Quick Sort different? (more difficult?)
 - Each recursive call of Quick Sort could have a different sized set of numbers to sort
 - Because the size of the sets is based on our partition element
 - If partition element is in the middle, each set has about half
 - Otherwise, one set is large and one is small

C

P

C

S

2

0

4

Quick Sort - **Analysis**

- **Best Case**

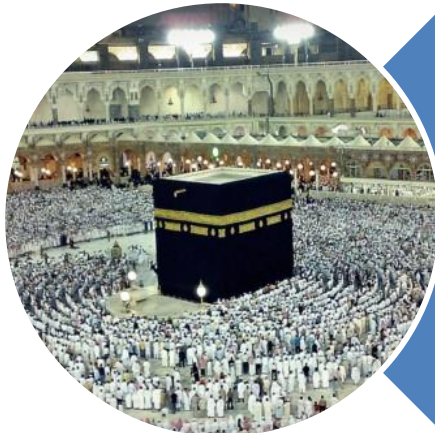
- The partition element is ALWAYS placed in the middle
- $T(n) = 2T(n/2) + n$
- $O(n \log_2 n)$

- **Worst Case**

- The partition element is placed either as first or last element
- $T(n) = T(n-1) + n$
- $O(n^2)$

- **Average Case**

- The partition element is placed anywhere in the arrays
- $T(n) = T(n/3) + T(2n/3) + n$
- $O(n \log_2 n)$



Allah (Alone) is Sufficient for us,
and He is the Best Disposer of
affairs (for us). **(Quran 3 : 173)**